
**RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE
SCIENTIFIQUE**



Université d'Alger 1 - Benyoucef Benkhedda

Report of Mini-Projet :
**RÉSEAU SOCIAL SYSTÈMES DE
RECOMMANDATION**

BENABDOUS Mohamed Mokhtar	Group 02
CHETTOUH Sadek El Amine	Group 01
OUAREZKI Oussama Abderahim	Group 02
MECHKAK Youcef Amine	Group 02

2^{ème} Année Master – Analyse et Sciences de Données

Année Universitaire 2024/2025

1. Introduction

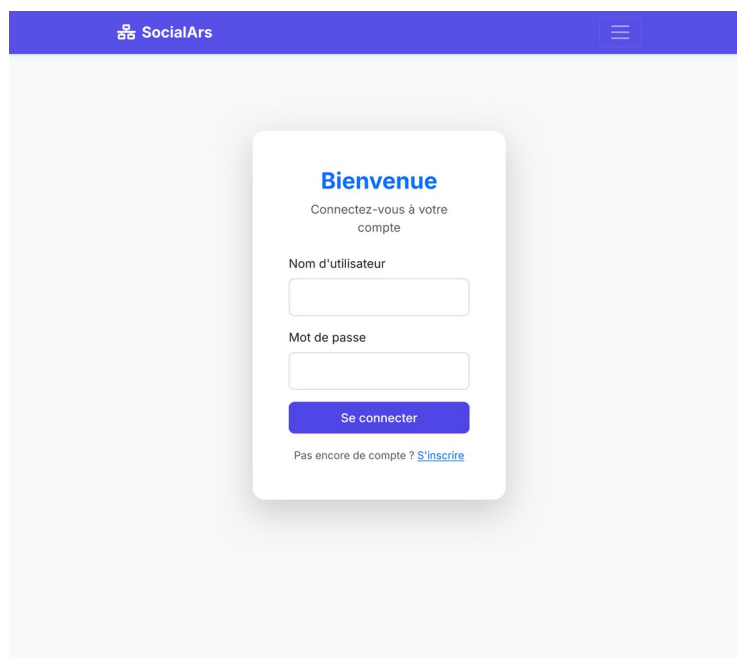
1.1 Contexte

Les réseaux sociaux occupent une place centrale dans le web moderne. Au-delà de simples plateformes d'échange, ils reposent sur des structures de données complexes (graphes) et des algorithmes avancés pour filtrer l'information. Dans le cadre du module "Analyse des Réseaux Sociaux", nous avons développé une application web complète intégrant ces concepts.

1.2 Objectifs

L'objectif principal était de concevoir et développer un mini-réseau social fonctionnel permettant :

1. La gestion d'une identité numérique (Profils).
2. L'interaction sociale (Amitiés, Likes, Commentaires).
3. L'implémentation de **systèmes de recommandation** basés sur la théorie des graphes et le filtrage collaboratif/hybride.



2. Modélisation et Conception

2.1 Le Graphe Social

Le cœur de notre application repose sur la modélisation des relations sous forme de graphe :

- **Nœuds (Nodes)** : Représentent les Utilisateurs.
- **Arêtes (Edges)** : Représentent les relations d'amitié.
 - Le graphe est **non-orienté** (l'amitié est réciproque).
 - Il est **dynamique** (création/suppression de liens).

2.2 Modèle de Données (Schéma Relationnel)

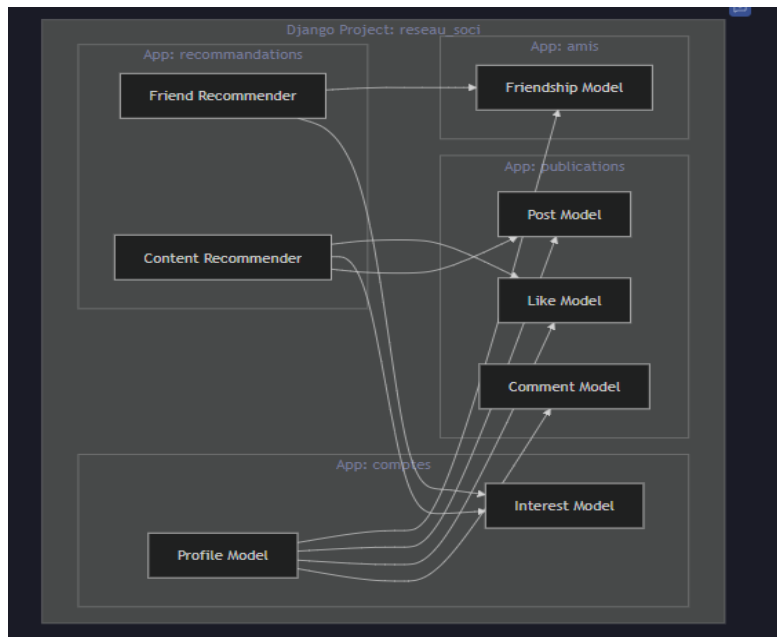
Nous avons utilisé une base de données relationnelle (SQLite/Django ORM) structurée autour de quatre entités principales :

1. **Utilisateur (Profile)** : Contient les informations biographiques et les **Centres d'Intérêt**.
 2. **Amitié (Friendship)** : Table de jointure reliant deux utilisateurs.
 3. **Publication (Post)** : Contenu textuel et multimédia.
 4. **Interaction (Like)** : Marque l'intérêt explicite.
-

3. Architecture Technique

3.1 Stack Technologique

- **Backend** : Python avec le framework **Django**.
- **Frontend** : HTML5, CSS3 (Bootstrap 5) et JavaScript.
- **Outils** : Git, VS Code.



4. Algorithmes de Recommandation

C'est la partie centrale du projet, exploitant les données du graphe social.

4.1 Recommandation d'Amis (Link Prediction)

L'objectif est de suggérer à un utilisateur **U** de nouvelles connexions potentielles. Nous avons implémenté un algorithme hybride combinant deux métriques.

A. Voisins Communs (Common Neighbors)

Basé sur la structure du graphe. Si l'utilisateur **U** et l'utilisateur **V** ne sont pas amis mais partagent beaucoup d'amis en commun, il est probable qu'ils se connaissent (phénomène de *Fermeture Triadique*).

Formule simplifiée :

$$\text{Score_Topo} = \text{Nombre d'amis que U et V ont en commun}$$

B. Similarité Homophilique

Basé sur les attributs des profils. On calcule la similarité (Jaccard) entre les ensembles de centres d'intérêt de U et de V.

Formule simplifiée :

$$\text{Score_Sim} = (\text{Intérêts communs}) / (\text{Total des intérêts de U et V})$$

Score Final :

Une pondération est appliquée pour obtenir le classement final :

$$\text{Score_Global} = (0.6 \times \text{Score_Topo}) + (0.4 \times \text{Score_Sim})$$

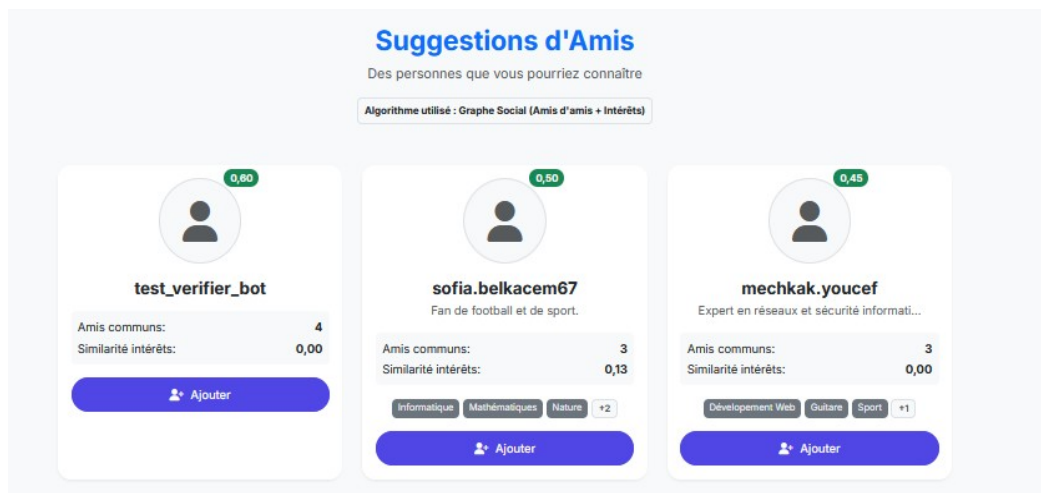


Figure 1 : Interface montrant les scores de recommandation calculés par l'algorithme.

4.2 Recommandation de Contenu

L'objectif est de proposer des posts pertinents. L'algorithme analyse trois dimensions :

1. **Match d'Intérêts** : Correspondance entre les tags du post et les intérêts de l'utilisateur.
2. **Popularité** : Les posts ayant beaucoup de likes sont favorisés.
3. **Proximité Sociale** : Les posts créés par les "amis d'amis" reçoivent un bonus.

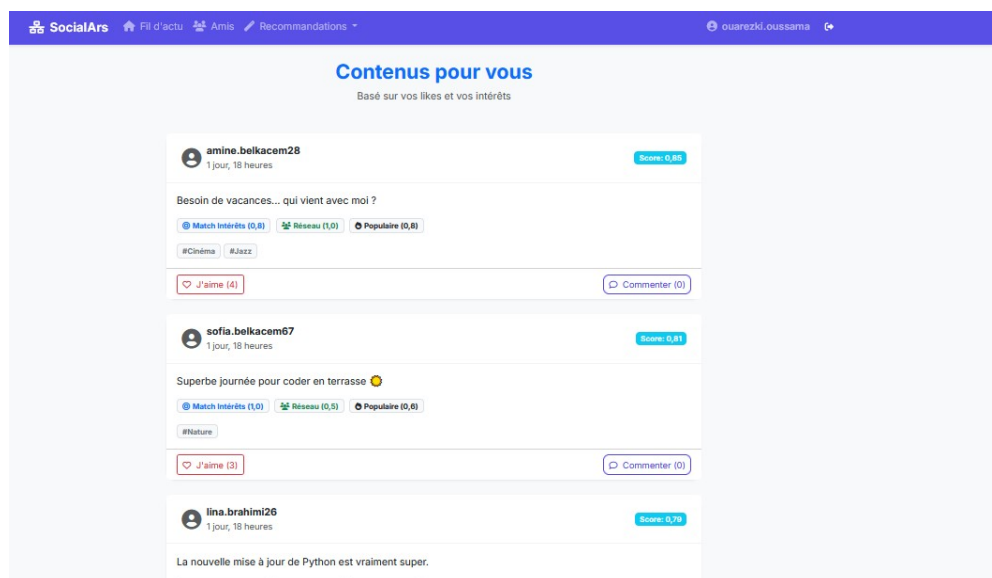


Figure 2 : Fil de recommandations mélangeant popularité et intérêts.

5. Fonctionnalités Implémentées

5.1 Expérience Utilisateur

- **Fil d'Actualité Riche** : Affichage fluide des posts avec images.

- **Interactivité** : Les "Likes" se font en temps réel (AJAX).
- **Système de Commentaires** : Intégré via des fenêtres pop-up.

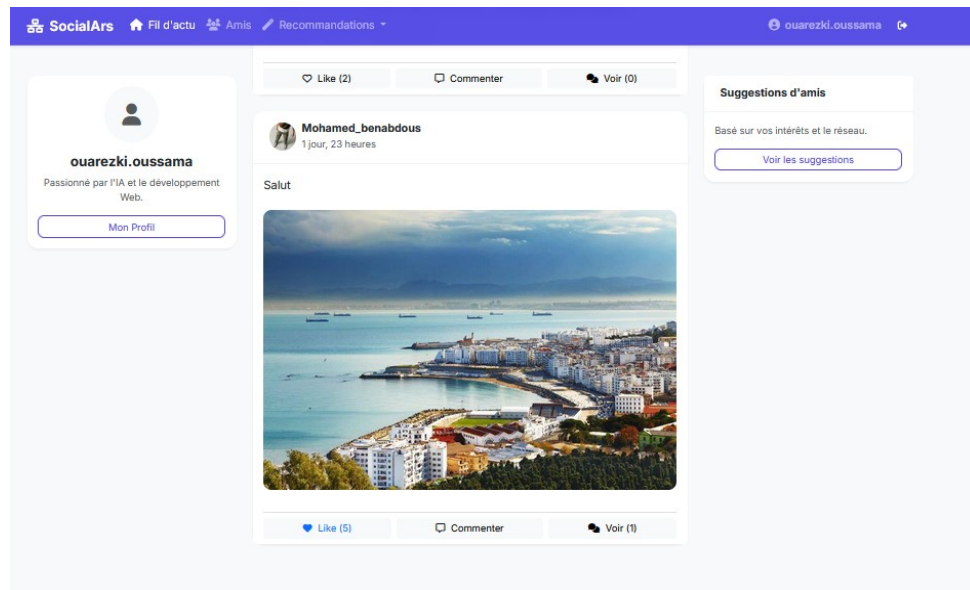


Figure 3 : Interface principale avec gestion des posts et commentaires.

5.2 Administration et Données

- **Peuplement Automatique** : Un script Python (`populate_db.py`) a été développé pour générer des données de test.
- **Interface Admin** : Utilisation de l'admin Django pour la supervision.

6. Conclusion

Ce projet nous a permis de mettre en pratique les concepts théoriques de l'Analyse des Réseaux Sociaux. Nous avons réussi à construire une plateforme fonctionnelle où la structure du graphe (les amitiés) influence directement l'expérience utilisateur.

Les tests effectués montrent que les suggestions sont pertinentes : l'algorithme identifie correctement les "clusters" d'amis et propose des contenus adaptés.